ASSIGNMENT - 4

High Performance Computing Lab (HPCL)

NAME	Yash Dhumal
ENROLLMENT	MITU22BTCS1017
SUBJECT	HPCL
ASSIGNMENT TITLE	ASSIGNMENT - 4

MPI Programming – Four Practical Programs

QUESTION 1

Write a simple MPI program to display a Hello World message from each process. The program should initialize MPI, obtain the rank of each process and the total number of processes, print the message, and finally finalize MPI.

Theory / Explanation

MPI (Message Passing Interface) is used to run a program using multiple processes. Each process has a unique rank, starting from 0. MPI_Comm_rank() gives the rank of the current process, while MPI_Comm_size() gives the total number of processes. When the program is executed with four processes, the ranks are 0, 1, 2 and 3.

In this program, MPI_Init() starts the MPI environment and MPI_Finalize() ends it. Each process prints its own rank and the total number of processes.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Variables for process rank and total number of processes
    int rank;
    int size;

    // Get the rank of the current process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

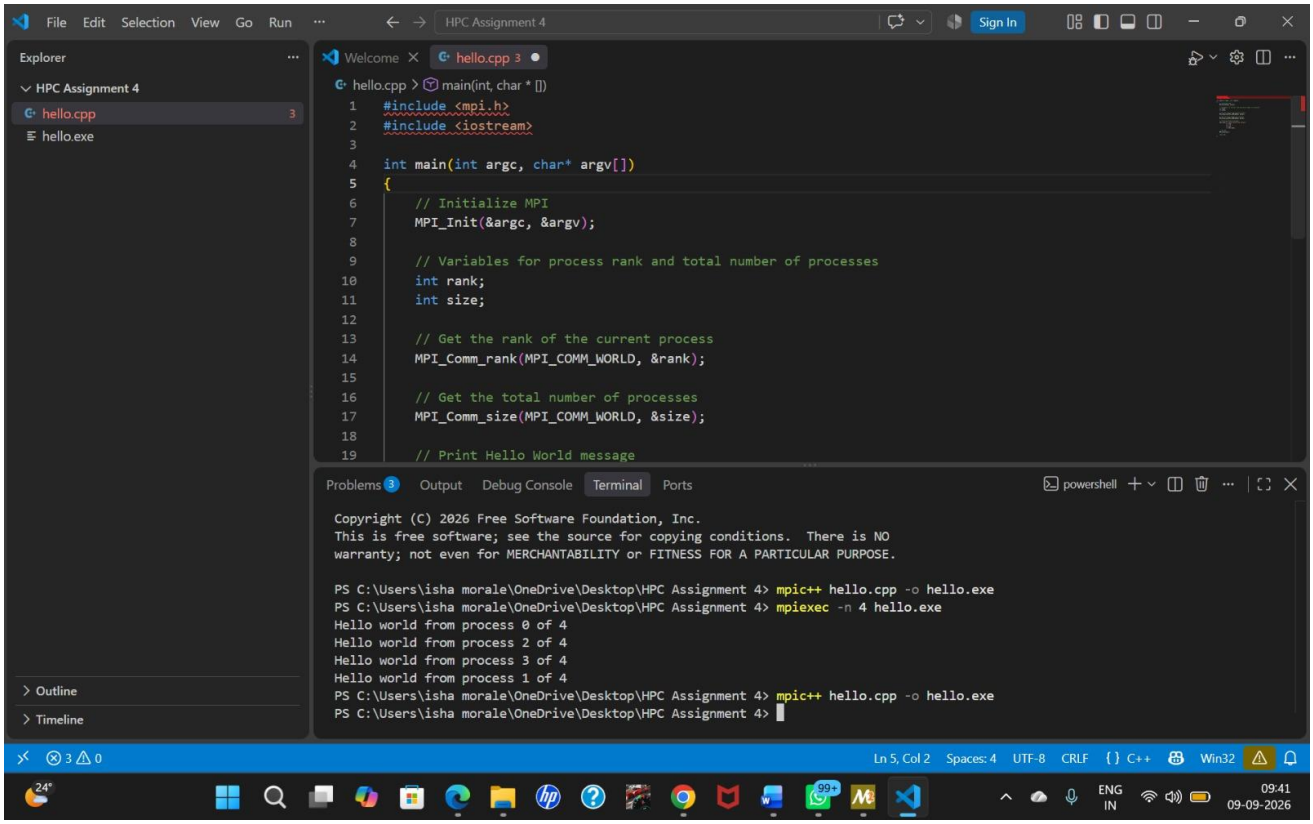
    // Get the total number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Print Hello World message
    std::cout << "Hello world from process "
              << rank
              << " of "
              << size
              << std::endl;

    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output:


File Edit Selection View Go Run
HPC Assignment 4
Sign In
Explorer
HPC Assignment 4
hello.cpp 3
hello.exe
Welcome
hello.cpp 3
hello.cpp > main(int, char * [])
#include <mpi.h>
#include <iostream>
int main(int argc, char* argv[])
{
// Initialize MPI
MPI_Init(&argc, &argv);
// Variables for process rank and total number of processes
int rank;
int size;
// Get the rank of the current process
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
// Get the total number of processes
MPI_Comm_size(MPI_COMM_WORLD, &size);
// Print Hello World message
Problems 3
Output
Debug Console
Terminal
Ports
powershell
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
PS C:\Users\isha morale\OneDrive\Desktop\HPC Assignment 4> mpic++ hello.cpp -o hello.exe
PS C:\Users\isha morale\OneDrive\Desktop\HPC Assignment 4> mpiexec -n 4 hello.exe
Hello world from process 0 of 4
Hello world from process 2 of 4
Hello world from process 3 of 4
Hello world from process 1 of 4
PS C:\Users\isha morale\OneDrive\Desktop\HPC Assignment 4> mpic++ hello.cpp -o hello.exe
PS C:\Users\isha morale\OneDrive\Desktop\HPC Assignment 4>
Outline
Timeline
Ln 5, Col 2
Spaces: 4
UTF-8
CRLF
C++
Win32
ENG
IN
09:41
09-09-2026

QUESTION 2

Write a program that reads an integer value from the terminal and distributes the value to all of the MPI processes. Each process should print its rank and the value it received. Values should be read until a negative integer is given as input.

Theory / Explanation

This program demonstrates broadcasting in MPI. Process 0 (the master process) reads an integer from the user. MPI_Bcast() then sends the same value from process 0 to every process in MPI_COMM_WORLD. Therefore, all processes receive the same value and print their rank along with that value.

The program keeps accepting values. If the value is negative, MPI_Bcast() distributes the negative value to all processes and every process exits the loop. A negative value is therefore used as the termination condition.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get process rank and total number of processes
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int value;

    while (true)
    {
        // Process 0 reads the value
        if (rank == 0)
        {
            std::cout << "Enter an integer (negative number to stop): ";
            std::cin >> value;
        }

        // Distribute the value to all processes
        MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);

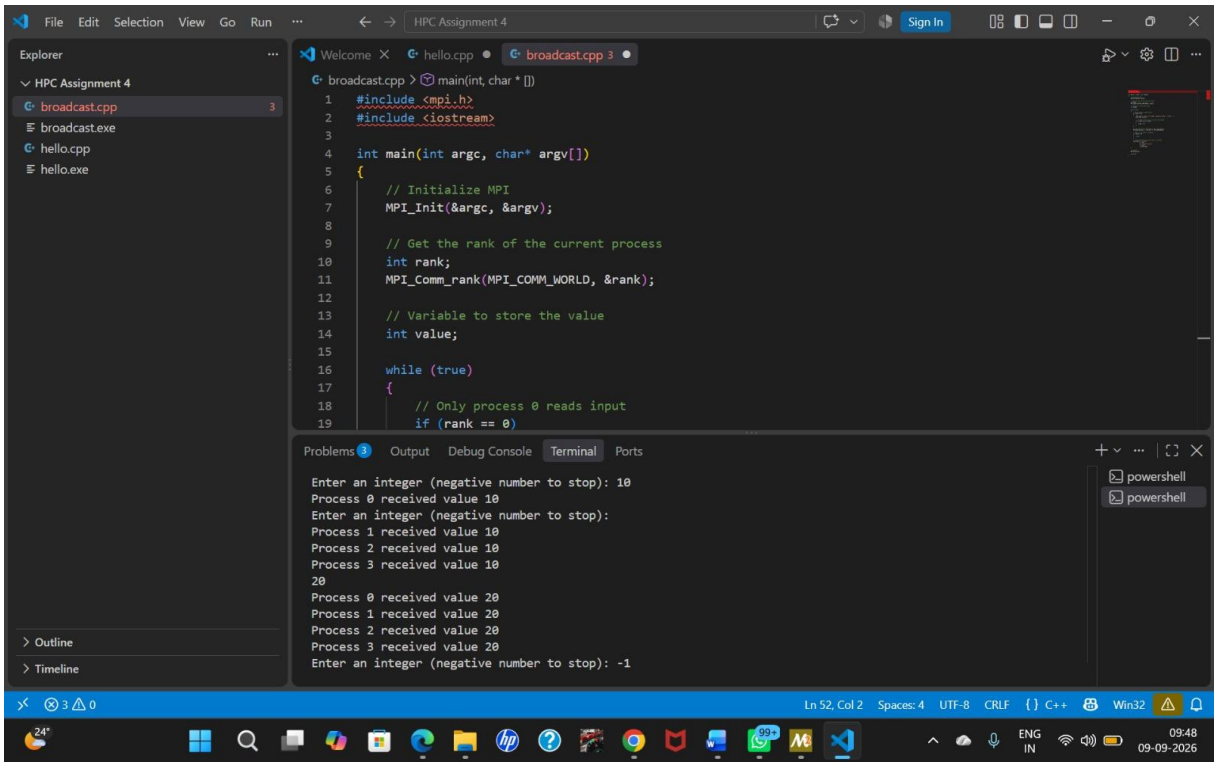
        // If the value is negative, stop all processes
        if (value < 0)
        {
            break;
        }

        // Each process prints its rank and the value received
        std::cout << "Process " << rank
                  << " received value " << value << std::endl;
    }
```

```
    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output



QUESTION 3

Write a program that takes data from process zero and sends it to all of the other processes by sending it in a ring. Process i should receive the data and send it to process i+1 until the last process is reached.

Theory / Explanation

A ring communication pattern passes data from one process to the next process in sequence. Process 0 first reads the value and sends it to process 1. Process 1 receives the value and sends it to process 2. This continues until the last process is reached.

MPI_Send() is used to send the value and MPI_Recv() is used to receive it. The destination of each process is its next rank. This demonstrates point-to-point communication and the ring topology in MPI.

Code

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and total number of processes
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int value;

    // Process 0 reads the data
    if (rank == 0)
    {
        std::cout << "Enter an integer: ";
        std::cin >> value;

        // Send data to process 1
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

        std::cout << "Process 0 sent value " << value
                  << " to process 1" << std::endl;
    }
    else
    {
        // Receive data from previous process
        MPI_Recv(&value, 1, MPI_INT, rank - 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        std::cout << "Process " << rank
                  << " received value " << value
                  << " from process " << rank - 1 << std::endl;

        // Send data to next process if it is not the last process
        if (rank < size - 1)
        {
```

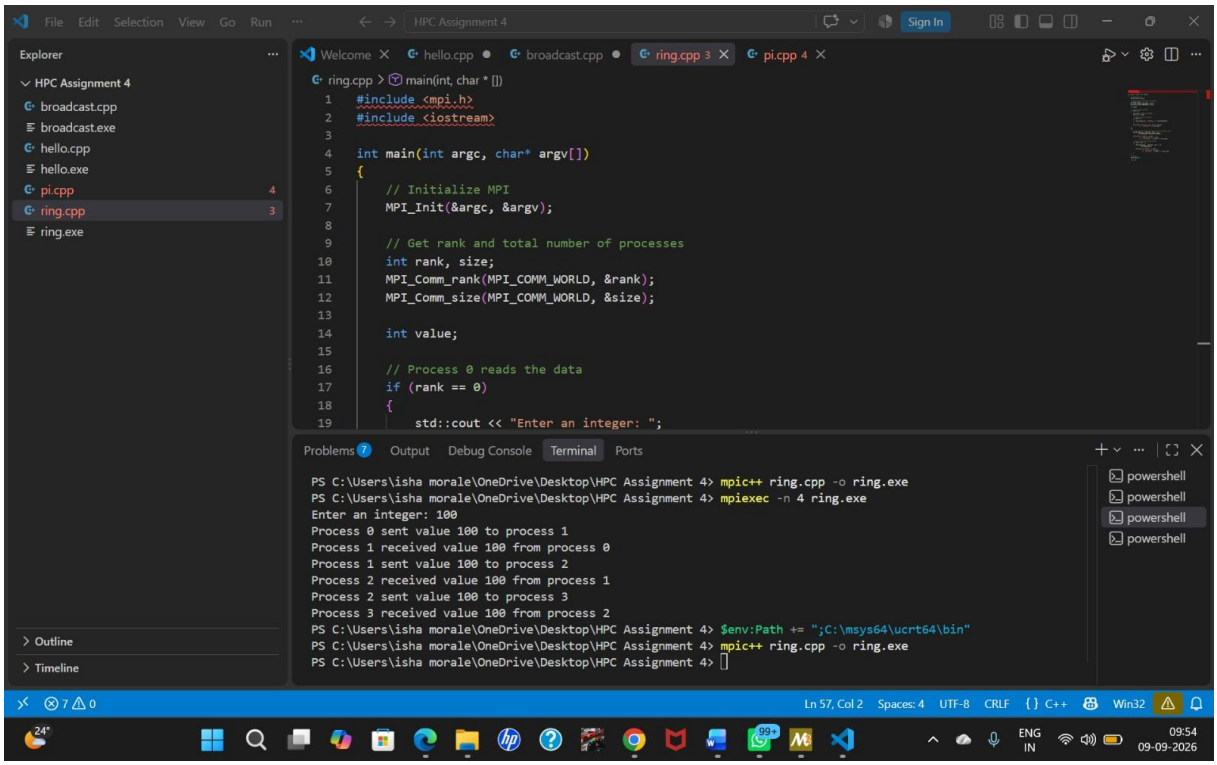
```
            MPI_Send(&value, 1, MPI_INT, rank + 1, 0,
                     MPI_COMM_WORLD);

            std::cout << "Process " << rank
                      << " sent value " << value
                      << " to process " << rank + 1 << std::endl;
        }
    }

    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output



QUESTION 4

Write a simple program to determine the value of pi. The method evaluates the integral of 4/(1+x*x) between 0 and 1. The integral is approximated by a sum of n intervals. The master process (rank 0) asks the user for the number of intervals, broadcasts this number to all other processes, and each process adds every nth interval. Finally, the partial sums are added using a reduction. Use MPI_Bcast and MPI_Reduce.

Theory / Explanation

The value of pi can be approximated by evaluating the integral of $4/(1+x^2)$ from 0 to 1. The interval from 0 to 1 is divided into n small intervals. The width of every interval is 1/n, and the contribution of an interval is calculated using $4/(1+x^2)$.

Process 0 asks the user for the number of intervals. MPI_Bcast() broadcasts this number to all processes so that every process knows the same value of n. The processes divide the work by calculating every size-th interval, starting at its own rank. Each process obtains a partial sum. MPI_Reduce() with MPI_SUM adds all partial sums and places the final result at process 0.

For example, with four processes, process 0 handles intervals 0, 4, 8, ...; process 1 handles 1, 5, 9, ...; process 2 handles 2, 6, 10, ...; and process 3 handles 3, 7, 11, ... This allows the numerical integration work to be performed in parallel.

Code

```
#include <mpi.h>
#include <iostream>
#include <iomanip>

int main(int argc, char* argv[])
{
    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get process rank and total number of processes
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Number of intervals
    int n;

    // Process 0 asks the user for the number of intervals
    if (rank == 0)
    {
        std::cout << "Enter the number of intervals: ";
        std::cin >> n;
    }

    // Broadcast n to all processes
    MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Calculate the width of each interval
    double h = 1.0 / static_cast<double>(n);
    double local_sum = 0.0;
```

```
    // Each process calculates every size-th interval
    for (int i = rank; i < n; i += size)
    {
        double x = h * (i + 0.5);
        local_sum += 4.0 / (1.0 + x * x);
    }

    // Multiply by interval width
    local_sum *= h;

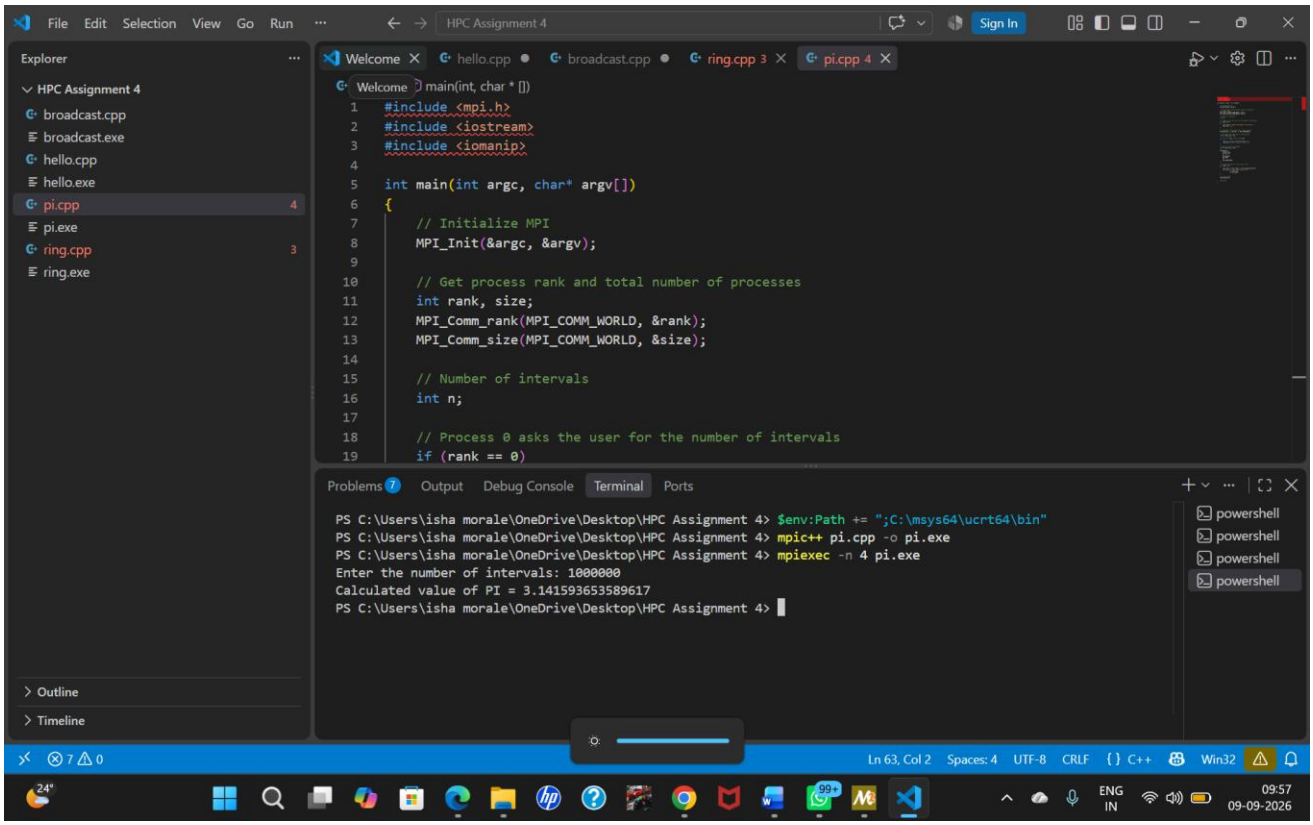
    // Reduce all partial sums to process 0
    double total_sum = 0.0;
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE,
               MPI_SUM, 0, MPI_COMM_WORLD);

    // Process 0 prints the final value of pi
    if (rank == 0)
    {
        std::cout << std::setprecision(15);
        std::cout << "Calculated value of PI = "
                  << total_sum << std::endl;
    }

    // End MPI
    MPI_Finalize();

    return 0;
}
```

Output



Conclusion

The four MPI programs demonstrate basic parallel programming concepts: process identification, broadcasting data to all processes, point-to-point communication in a ring, and parallel numerical computation using reduction. MPI_Bcast was used for distributing common data, while MPI_Reduce was used to combine partial results. The programs were compiled with mpic++ and executed using mpiexec with four processes.